

Software Requirements Specification (SRS) for SENA Network

Document Version: 1.0.0

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to detail the functional and non-functional requirements for the SENA Network, a Layer 2 (L2) modular infrastructure. This document serves as a foundational guide for development, testing, and deployment, ensuring all stakeholders have a clear understanding of the system to be built.

1.2 Scope

The SENA Network will provide a high-performance, secure, and user-friendly L2 blockchain solution built on the Sovereign SDK (Rust) and leveraging RocksDB for state management. It will integrate with the Aptos Layer 1 (L1) for security settlement and will feature protocol-level support for keyless authentication, social identity mapping, multi-asset gas payments, and optimistic governance. This SRS covers the requirements for the core L2 protocol and its primary modules.

1.3 Definitions, Acronyms, and Abbreviations

- **L2:** Layer 2 blockchain, built on top of an existing Layer 1 blockchain.
- **L1:** Layer 1 blockchain, the base layer for security and finality (in this case, Aptos).
- **SENA:** The name of the Layer 2 network.
- **Sovereign SDK:** A framework for building modular rollups in Rust.
- **RocksDB:** A high-performance embedded key-value store.
- **NOMT:** Nearly-Optimal Merkle Trie, an optimized Merkle tree implementation.
- **Aptos:** The Layer 1 blockchain providing security and settlement for SENA.
- **OIDC:** OpenID Connect, an authentication layer on top of OAuth 2.0.
- **JWT:** JSON Web Token, a compact, URL-safe means of representing claims between two parties.

- **ZKP**: Zero-Knowledge Proof, a method by which one party (the prover) can prove to another party (the verifier) that a given statement is true, without conveying any information apart from the fact that the statement is indeed true.
- **EPK**: Ephemeral Public Key.
- **PRD**: Product Requirements Document.
- **SRS**: Software Requirements Specification.
- **SDD**: Software Design Description.

1.4 References

- [1] SENA Network: Layer 2 Modular Infrastructure Documentation (Previous Stakeholder Document)
- [2] Sovereign SDK. *Sovereign Labs*. Available at: <https://www.sovereign.xyz/>
- [3] RocksDB. *GitHub*. Available at: <https://github.com/facebook/rocksdb>
- [4] Aptos Keyless Accounts. *Aptos Docs*. Available at: <https://aptos.dev/concepts/accounts/#keyless-accounts>

2. Overall Description

2.1 Product Perspective

SENA Network is a standalone L2 blockchain that operates as a client-server application, with its security and data availability rooted in the Aptos L1. It is not a smart contract on Aptos L1 but rather an independent chain that periodically commits its state to Aptos L1. This architecture allows SENA to achieve high transaction throughput and low latency while inheriting the robust security guarantees of Aptos.

2.2 Product Functions

The primary functions of the SENA Network include:

- Processing and executing L2 transactions.
- Managing user accounts with Web2-style keyless authentication.
- Mapping human-readable social identifiers to blockchain addresses.
- Facilitating gas payments using stablecoins.
- Supporting an optimistic governance mechanism.
- Committing L2 state roots to Aptos L1 for security and finality.

2.3 User Characteristics

- **End-Users:** Expect a seamless, intuitive experience akin to Web2 applications, without needing to manage seed phrases or native gas tokens. They will interact with applications built on SENA.
- **Application Developers:** Require a robust, well-documented, and easy-to-integrate platform for building consumer-facing dApps. They will interact with SENA's RPC endpoints and SDKs.
- **Node Operators:** Responsible for running SENA sequencer nodes. They should have basic system administration skills and an understanding of blockchain node operation.

2.4 General Constraints

- **Technology Stack:** Must be built using Rust and Sovereign SDK.
- **Database:** Must utilize RocksDB for persistent storage, optimized with NOMT.
- **L1 Integration:** Must integrate with Aptos L1 for security settlement and state anchoring.
- **Performance:** Must support high transaction throughput suitable for consumer applications.
- **Security:** Must achieve "Aptos-Level Secure" as defined in the stakeholder documentation.
- **Resource Efficiency:** Must be capable of running on low-end server hardware for sequencer nodes.

2.5 Assumptions and Dependencies

- Aptos L1 remains a stable and secure blockchain.
- The Aptos public, decentralized ZK prover network is available and reliable for offloading ZKP generation.
- Sovereign SDK continues to be actively developed and maintained.
- Rust ecosystem tools and libraries remain stable and compatible.

3. Specific Requirements

3.1 Functional Requirements

3.1.1 Module 1: Protocol-Enforced Keyless Authentication Engine

- **REQ-AUTH-001:** The system SHALL natively process OpenID Connect (OIDC) identities as standard account signers at the protocol layer.
- **REQ-AUTH-002:** The system SHALL derive deterministic L2 addresses using an application-scoped identifier (`$aud`), a user identifier (`$sub`), and an obfuscating pepper value.
- **REQ-AUTH-003:** The system SHALL accept transactions signed by a short-lived Ephemeral Public Key (EPK).
- **REQ-AUTH-004:** The system SHALL verify the validity of an EPK against a Zero-Knowledge Proof (ZKP) that references a valid Web2 JWT signature from an approved Identity Provider (e.g., Google or Apple OAuth).
- **REQ-AUTH-005:** The system SHALL NOT calculate heavy ZK proof constraints locally; it SHALL ingest pre-computed proofs generated by external client-side utilities or decentralized prover networks.
- **REQ-AUTH-006:** The system SHALL perform lightweight cryptographic verification checks of ZK proofs during block execution.

3.1.2 Module 2: The Social Connect Identity Index

- **REQ-SOCIAL-001:** The system SHALL maintain an embedded system ledger connecting human-readable identifiers directly to SENA network addresses.
- **REQ-SOCIAL-002:** The system SHALL store identifiers as unique cryptographic hashes (`SHA-256(Raw Identifier + Network Salt)`) to ensure privacy.
- **REQ-SOCIAL-003:** The system SHALL NOT expose plain text personal information (phone numbers, emails, handles) directly in the global state database.
- **REQ-SOCIAL-004:** The system SHALL provide an RPC endpoint for external financial or payment software to perform lookups to resolve a destination address instantly from a human-readable identifier.
- **REQ-SOCIAL-005:** The system SHALL support mapping a single SENA address to multiple verified Web2 social channels simultaneously.

3.1.3 Module 3: Multi-Asset Gas Paymaster Engine

- **REQ-GAS-001:** The system SHALL support transaction gas payment via an approved whitelist of stablecoins.
- **REQ-GAS-002:** The system SHALL remove the requirement for users to hold a native network utility token for routine actions.
- **REQ-GAS-003:** The network state machine SHALL handle an adjustable system ledger tracking permitted gas assets (e.g., native bridged USDC, USDT, and EUR-equivalent stablecoins).
- **REQ-GAS-004:** When an application transaction is processed, the sequencer SHALL evaluate the current gas cost.
- **REQ-GAS-005:** The sequencer SHALL look up the real-time fee rate for the chosen stablecoin.
- **REQ-GAS-006:** The sequencer SHALL extract the fraction of a penny directly out of the user's transaction payload or account balance as payment.
- **REQ-GAS-007:** Collected stablecoin fees SHALL be directed into a system-controlled vault.
- **REQ-GAS-008:** The sequencer SHALL handle converting a portion of collected assets into native APT tokens via the Aptos L1 bridge contract.
- **REQ-GAS-009:** The system SHALL continuously fund the L2's cryptographic checkpoint commitments on the Layer 1 mainnet using converted APT tokens.

3.1.4 Module 4: Optimistic L2 Governance Layer

- **REQ-GOV-001:** The system SHALL support an off-chain aggregation mechanism for token holders to vote on parameter revisions by staking the network's asset (**\$SENA**).
- **REQ-GOV-002:** Once a vote passes off-chain, the structured result payload SHALL be signed by a designated Governance Multi-Signature Council.
- **REQ-GOV-003:** The SENA node runtime SHALL include an administrative module to ingest the council-signed payload.
- **REQ-GOV-004:** The administrative module SHALL authenticate the cryptographic signatures in milliseconds.
- **REQ-GOV-005:** The administrative module SHALL instantly apply updates to system variables (e.g., whitelisting new stablecoins, adjusting network fee schedules) based on authenticated payloads.

3.1.5 Core L2 Protocol Requirements

- **REQ-CORE-001:** The L2 sequencer SHALL process transactions locally.
- **REQ-CORE-002:** The sequencer SHALL periodically submit a State Root (Merkle Root) representing all accounts, social links, and balances to an Aptos L1 Move contract.
- **REQ-CORE-003:** The system SHALL ensure that once a State Root is committed to Aptos Mainnet, it becomes immutable.
- **REQ-CORE-004:** Users SHALL be able to use on-chain proof from Aptos L1 to withdraw their funds directly from the L1 bridge in case of L2 server compromise.

3.2 Non-Functional Requirements

3.2.1 Performance Requirements

- **NFR-PERF-001:** The L2 network SHALL achieve transaction throughput suitable for consumer applications (e.g., >1,000 transactions per second).
- **NFR-PERF-002:** Transaction finality on the L2 SHALL be achieved within seconds.
- **NFR-PERF-003:** State Root commitments to Aptos L1 SHALL occur every few minutes.
- **NFR-PERF-004:** ZK proof verification SHALL require minimal CPU power to maintain high performance on low-end server infrastructure.

3.2.2 Security Requirements

- **NFR-SEC-001:** The system SHALL inherit security from Aptos L1 through cryptographic commitments of the L2 state root.
- **NFR-SEC-002:** User funds SHALL be recoverable directly from Aptos L1 even if the L2 server is compromised.
- **NFR-SEC-003:** All sensitive data, especially personal identifiers, SHALL be stored in a privacy-preserving manner (e.g., cryptographic hashes).
- **NFR-SEC-004:** The Keyless JWT Validation Architecture SHALL prevent exposure of personal information while linking Web2 IDs to blockchain addresses.
- **NFR-SEC-005:** The system SHALL prevent replay attacks for EPK-signed transactions.
- **NFR-SEC-006:** All cryptographic operations SHALL use industry-standard, audited algorithms and libraries.

3.2.3 Reliability Requirements

- **NFR-REL-001:** The L2 sequencer SHALL maintain high uptime and availability.
- **NFR-REL-002:** The system SHALL gracefully handle network partitions and temporary L1 unavailability.
- **NFR-REL-003:** Data persistence SHALL be guaranteed through RocksDB, ensuring no loss of state.

3.2.4 Maintainability Requirements

- **NFR-MAINT-001:** The codebase SHALL be written in Rust, adhering to idiomatic Rust practices and coding standards.
- **NFR-MAINT-002:** The system SHALL be modular, allowing for independent updates and upgrades of its components (e.g., Keyless Auth Engine, Social Connect Index).
- **NFR-MAINT-003:** Comprehensive logging and monitoring capabilities SHALL be integrated to facilitate debugging and operational oversight.

3.2.5 Portability Requirements

- **NFR-PORT-001:** The SENA L2 node software SHALL be deployable on standard Linux-based server environments.
- **NFR-PORT-002:** The system SHALL be compatible with various cloud providers and on-premise deployments.

4. Appendices

4.1 Stakeholder Documentation Excerpt: Network Security Model

SENA is designed to achieve “Aptos-Level Secure” by inheriting the robust security guarantees of the Aptos L1, even when operating on resource-constrained infrastructure. This approach ensures that user funds remain safe from compromise, even if the L2 server itself is physically breached.

A. Inherited Security via Cryptographic Commitment

SENA's L2 server functions as a single sequencer, processing transactions locally. To ensure the integrity and immutability of the L2 state, the sequencer periodically submits a **State Root**

(a Merkle Root representing all accounts, social links, and balances) to a dedicated Aptos L1 Move contract.

The Security Guarantee: Once this State Root is committed to the Aptos Mainnet, it becomes an immutable record. In the event of a compromise or failure of the L2 server, the history of asset ownership is indelibly recorded on the multi-billion-dollar Aptos L1 validator network. Users can leverage this on-chain proof to directly withdraw their funds from the L1 bridge, ensuring no loss of assets.

B. Secure Keyless JWT Validation Architecture

SENA integrates with **Aptos Keyless accounts** to provide a secure and user-friendly authentication mechanism. This system utilizes Zero-Knowledge (ZK) Proofs to link a user's Web2 identity (e.g., Google JWT) to their blockchain address without exposing sensitive personal information.

Implementation Details:

- **Client-Side Key Pair Generation:** When a user initiates a login via a Web2 provider (e.g., Google, Apple), the client application generates an ephemeral key pair.
- **ZK Proof Request:** The client then requests a ZK proof from a dedicated Prover service. SENA leverages Aptos's public, decentralized prover network for this purpose, offloading the computationally intensive ZK circuit computations from the L2 server.
- **On-Chain Verification:** The SENA L2 node incorporates Aptos's open-source signature verification code, written natively in Rust. Upon receiving a transaction, the sequencer performs a lightweight verification of the pre-computed ZK proof. This verification process requires minimal CPU power compared to generating the proof, allowing SENA to maintain high performance on low-end server infrastructure.